



Automation

Jenkins Advanced



Outline

1**Pendahuluan****2****Integrasi Jenkins dengan GitHub****3****Webhook untuk Automasi
Deployment****4****Hands-on**

1

Pendahuluan

Jenkins Advanced



Apa itu **Continuous Integration (CI)**?

Definisi

Continuous Integration adalah praktik pengembangan perangkat lunak di mana anggota tim mengintegrasikan kode mereka secara teratur, biasanya beberapa kali sehari. Setiap integrasi diverifikasi oleh build otomatis untuk mendeteksi kesalahan secepat mungkin.

Manfaat

- Deteksi bug lebih awal
- Mengurangi konflik integrasi
- Meningkatkan kualitas kode
- Mempercepat siklus rilis



Apa itu **Continuous Delivery/Deployment (CD)**?

Continuous Delivery memastikan bahwa perangkat lunak dapat dirilis kapan saja, sementara Continuous Deployment secara otomatis menerapkan setiap perubahan yang lulus semua tahap pipeline produksi.



Develop

Pengembangan
fitur baru



Test

Pengujian
otomatis



Build

Pembuatan
artefak



Deploy

Penerapan ke
produksi

Manfaat Utama CI/CD

Pengembangan Lebih Cepat

Otomatisasi proses build dan deployment mempercepat siklus pengembangan dan memungkinkan rilis lebih sering.

Peningkatan Kualitas

Pengujian otomatis dan deteksi bug dini membantu meningkatkan kualitas kode dan mengurangi masalah di produksi.

Kolaborasi Lebih Baik

Tim dapat bekerja secara paralel dan mengintegrasikan perubahan dengan lebih efisien, meningkatkan kolaborasi.

Umpan Balik Cepat

Pengembang mendapatkan umpan balik instan tentang perubahan mereka, memungkinkan perbaikan cepat.

Peran Git dalam CI/CD

Version Control

Git melacak perubahan kode dan memungkinkan kolaborasi antar pengembang.



Branching

Memungkinkan pengembangan fitur paralel tanpa mengganggu kode utama.



Git

History

Menyimpan riwayat lengkap perubahan kode untuk audit dan rollback.



Merging

Mengintegrasikan perubahan dari berbagai branch ke dalam kode utama.



Peran GitHub dalam CI/CD

Hosting Repository

Platform berbasis cloud untuk menyimpan dan mengelola repositori Git.

Kolaborasi

Fitur pull request, issue tracking, dan code review untuk kolaborasi tim.

Integrasi

Webhook dan API untuk integrasi dengan alat CI/CD seperti Jenkins.

Keamanan

Fitur keamanan seperti pemindaian kode dan manajemen akses.

Peran Jenkins dalam CI/CD



Automation Server

Jenkins adalah server otomatisasi open-source yang memungkinkan pengembang untuk membangun, menguji, dan men-deploy perangkat lunak secara otomatis.



Pipeline

Jenkins Pipeline menyediakan set alat untuk memodelkan proses pengiriman perangkat lunak sederhana hingga kompleks sebagai kode.



Plugin Ecosystem

Ekosistem plugin yang luas memungkinkan integrasi dengan berbagai alat dan platform, termasuk GitHub.

**2**

Integrasi Jenkins dengan GitHub

Jenkins Advanced

Prasyarat Integrasi Jenkins-GitHub

Akun GitHub dan Repository

Anda memerlukan akun GitHub dan repositori yang akan digunakan untuk integrasi. Repository ini akan berisi kode sumber dan Jenkinsfile untuk pipeline CI/CD.

Jenkins Terinstal dan Berjalan

Jenkins harus sudah terinstal dan berjalan di server Anda. Pastikan Anda memiliki akses administrator ke instans Jenkins untuk menginstal plugin dan mengkonfigurasi sistem.

Koneksi Internet untuk Jenkins

Server Jenkins Anda harus memiliki koneksi internet untuk berkomunikasi dengan GitHub. Jika Jenkins berada di belakang firewall, pastikan port yang diperlukan terbuka.

Instalasi Plugin **GitHub** di **Jenkins**

Buka Manage Jenkins

Navigasi ke dashboard Jenkins dan klik "Manage Jenkins" di menu sebelah kiri.

Buka Manage Plugins

Klik "Manage Plugins" untuk membuka halaman pengelolaan plugin.

Cari GitHub Plugin

Di tab "Available", cari "GitHub Integration Plugin" menggunakan kotak pencarian.

Instal Plugin

Centang plugin yang diperlukan dan klik "Install without restart" atau "Download now and install after restart".

Pentingnya **Plugin GitHub**

Komunikasi Jenkins-GitHub

Plugin GitHub memungkinkan Jenkins berkomunikasi dengan GitHub melalui API GitHub. Ini memungkinkan Jenkins untuk mengambil kode dari repositori, memantau perubahan, dan merespons peristiwa GitHub seperti push atau pull request.

Fitur Utama

- Integrasi dengan GitHub Webhook
- Dukungan untuk GitHub OAuth
- Kemampuan untuk memperbarui status commit
- Integrasi dengan GitHub Pull Requests



Jenis Kredensial **GitHub** di **Jenkins**

Username/ Password

Kredensial dasar yang terdiri dari nama pengguna GitHub dan kata sandi. Metode ini sederhana tetapi kurang aman dibandingkan dengan metode lainnya dan tidak direkomendasikan untuk penggunaan jangka panjang.

SSH Key

Menggunakan pasangan kunci SSH untuk autentikasi. Lebih aman daripada username/password dan ideal untuk akses jangka panjang ke repository GitHub.

Access Token

Personal Access Token (PAT) dari GitHub yang memberikan akses tertentu ke API GitHub. Metode yang direkomendasikan untuk integrasi Jenkins-GitHub.

Mengapa **Kredensial Diperlukan?**

Akses Repository Privat

Kredensial diperlukan untuk mengakses repository privat di GitHub yang tidak tersedia untuk umum.

Operasi Penulisan

Untuk operasi yang memerlukan akses tulis ke repository, seperti memperbarui status commit atau membuat komentar pada pull request.

Akses API

Untuk mengakses API GitHub dengan batas rate yang lebih tinggi dan fitur tambahan yang tidak tersedia untuk pengguna anonim.

Konfigurasi Webhook

Untuk membuat dan mengelola webhook di repository GitHub secara programatis melalui Jenkins.

Membuat Personal Access Token di GitHub

Buka Settings di GitHub

Klik profil Anda di kanan atas, lalu pilih "Settings" dari dropdown menu.

Akses Developer Settings

Scroll ke bawah dan klik "Developer settings" di sidebar kiri.

Buat Personal Access Token

Pilih "Personal access tokens", lalu klik "Generate new token". Berikan nama deskriptif dan pilih scope yang sesuai (minimal "repo" untuk akses repository).

Simpan Token dengan Aman

Setelah token dibuat, salin dan simpan di tempat yang aman. Token hanya ditampilkan sekali dan tidak dapat diambil lagi.

Menambahkan Kredensial di Jenkins

Buka Manage Jenkins

Navigasi ke dashboard Jenkins dan klik "Manage Jenkins" di menu sebelah kiri.

Akses Manage Credentials

Klik "Manage Credentials" untuk membuka halaman pengelolaan kredensial.

Pilih Domain

Klik pada domain kredensial yang sesuai (biasanya "global") atau buat domain baru jika diperlukan.

Tambahkan Kredensial

Klik "Add Credentials", pilih jenis "Secret text", masukkan token GitHub Anda di field "Secret", berikan ID dan deskripsi, lalu klik "OK".

Pentingnya Pipeline as Code dengan Jenkinsfile

Apa itu Pipeline as Code?

Pipeline as Code adalah pendekatan di mana konfigurasi pipeline CI/CD didefinisikan sebagai kode dan disimpan bersama dengan kode sumber aplikasi. Dalam Jenkins, ini diimplementasikan menggunakan Jenkinsfile.

Manfaat

- Versi pipeline disimpan bersama kode
- Dapat diaudit dan ditinjau seperti kode lainnya
- Dapat diuji dan divalidasi
- Mendukung evolusi pipeline seiring waktu



Contoh Jenkinsfile Sederhana

```
// Jenkinsfilepipeline { agent any stages { stage('Checkout') {  
steps { // Ini akan otomatis checkout repository Jenkinsfile  
berada script { checkout scm } } }  
stage('Hello GitHub') { steps { echo 'Halo dari Jenkins,  
GitHub!' } } }}
```

Jenkinsfile ini mendefinisikan pipeline sederhana dengan dua tahap: "Checkout" untuk mengambil kode dari repositori, dan "Hello GitHub" yang hanya menampilkan pesan. Ini adalah titik awal yang baik untuk pipeline Jenkins Anda.



3

Webhook untuk Automasi Deployment

Jenkins Advanced



Apa itu Webhook?

Definisi

Webhook adalah mekanisme yang memungkinkan satu aplikasi untuk memberikan informasi real-time ke aplikasi lain ketika suatu peristiwa terjadi. Ini adalah cara untuk aplikasi berkomunikasi secara otomatis tanpa perlu polling terus-menerus.

Cara Kerja

Ketika peristiwa tertentu terjadi di aplikasi sumber (misalnya, push ke repository GitHub), aplikasi tersebut mengirim HTTP POST request dengan payload yang berisi informasi tentang peristiwa tersebut ke URL yang telah ditentukan di aplikasi tujuan (misalnya, Jenkins).



Perbedaan antara Polling dan Webhook

Aspek	Polling	Webhook
Inisiasi	Jenkins secara berkala memeriksa GitHub untuk perubahan	GitHub memberi tahu Jenkins ketika perubahan terjadi
Efisiensi	Kurang efisien, menggunakan sumber daya meskipun tidak ada perubahan	Lebih efisien, hanya menggunakan sumber daya ketika ada perubahan
Latensi	Lebih tinggi, tergantung pada interval polling	Lebih rendah, hampir real-time
Konfigurasi	Lebih sederhana, hanya perlu konfigurasi di Jenkins	Lebih kompleks, memerlukan konfigurasi di GitHub dan Jenkins

Manfaat Webhook untuk **Automasi Real-time**

Respons Instan

Build dimulai segera setelah perubahan kode, tanpa penundaan yang disebabkan oleh interval polling.

Efisiensi Sumber Daya

Mengurangi beban pada server Jenkins dan API GitHub karena tidak perlu polling terus-menerus.

Webhook

Skalabilitas

Dapat menangani banyak repository tanpa menambah beban polling yang signifikan.

Umpan Balik Lebih Cepat

Pengembang mendapatkan umpan balik lebih cepat tentang perubahan mereka, mempercepat siklus pengembangan.

Cara Kerja Webhook antara **GitHub** dan **Jenkins**

Peristiwa di GitHub

Pengembang melakukan push ke repository atau membuat pull request, memicu peristiwa di GitHub.

Webhook Dipicu

GitHub mendeteksi peristiwa dan mengirim HTTP POST request dengan payload JSON ke URL Webhook Jenkins yang terdaftar.

Jenkins Menerima Payload

Jenkins menerima request dan memproses payload untuk mengekstrak informasi tentang peristiwa.

Build Dimulai

Jenkins memicu build job yang sesuai berdasarkan informasi dalam payload.

Payload Webhook **GitHub**

Struktur Payload

Payload Webhook GitHub adalah dokumen JSON yang berisi informasi rinci tentang peristiwa yang terjadi. Ini mencakup data seperti repositori yang terkena dampak, commit yang terlibat, pengguna yang memicu peristiwa, dan detail spesifik tentang peristiwa itu sendiri.

Informasi Penting dalam Payload

- Jenis peristiwa (push, pull request, dll.)
- Repositori dan branch yang terkena dampak
- Commit yang terlibat (SHA, pesan, penulis)
- Pengguna yang memicu peristiwa
- Timestamp peristiwa

Konfigurasi Webhook di GitHub

Buka Pengaturan Repository

Navigasi ke repository GitHub Anda dan klik tab "Settings".

Akses Menu Webhooks

Klik "Webhooks" di sidebar kiri untuk membuka halaman pengelolaan webhook.

Tambahkan Webhook Baru

Klik tombol "Add webhook" untuk membuat webhook baru.

Konfigurasi Webhook

Masukkan URL Jenkins Anda dengan path `/github-webhook/`, pilih "application/json" sebagai Content type, dan pilih peristiwa yang akan memicu webhook.

Webhook Secret untuk Keamanan

Apa itu Webhook Secret?

Webhook Secret adalah string rahasia yang digunakan untuk mengamankan komunikasi antara GitHub dan Jenkins. GitHub menggunakan secret ini untuk menandatangani payload webhook, dan Jenkins memverifikasi tanda tangan untuk memastikan bahwa payload benar-benar berasal dari GitHub.

Manfaat Menggunakan Secret

- Mencegah serangan man-in-the-middle
- Memverifikasi keaslian payload
- Mencegah pemalsuan permintaan webhook
- Meningkatkan keamanan keseluruhan integrasi



Konfigurasi Job Jenkins untuk Menerima Webhook

Buat New Item di Jenkins

Di dashboard Jenkins, klik "New Item", masukkan nama untuk proyek baru, dan pilih "Pipeline" sebagai jenis proyek.

Konfigurasi GitHub Project

Di halaman konfigurasi, centang "GitHub project" dan masukkan URL repository GitHub Anda (misalnya, <https://github.com/username/repository>).

Aktifkan Trigger Webhook

Di bagian "Build Triggers", centang "GitHub hook trigger for GITScm polling" untuk memungkinkan Jenkins merespons webhook dari GitHub.

Konfigurasi Pipeline

Di bagian "Pipeline", pilih "Pipeline script from SCM", pilih "Git" sebagai SCM, masukkan URL repository, dan tentukan branch dan path ke Jenkinsfile.

Konfigurasi Pipeline **Script from SCM**



Pilih SCM

Di bagian "Pipeline" dari konfigurasi job, pilih "Pipeline script from SCM" dan kemudian pilih "Git" dari dropdown SCM.



Konfigurasi Repository

Masukkan URL repositori GitHub Anda dan pilih kredensial yang sesuai jika repository privat. Anda dapat menggunakan format HTTPS atau SSH tergantung pada jenis kredensial.



Branch dan Path Jenkinsfile

Tentukan branch yang akan dipantau (biasanya "main" atau "master") dan path ke Jenkinsfile di repositori (biasanya cukup "Jenkinsfile" jika berada di root).

4

Hands-on

Jenkins Advanced



Langkah-langkah **Hands-on**

Clone Repository

Clone repository contoh ke mesin lokal Anda menggunakan perintah git clone.

Buat Jenkins Project

Buat proyek Pipeline baru di Jenkins dan konfigurasi untuk menggunakan repositori GitHub Anda.

Konfigurasi Webhook

Tambahkan Webhook di pengaturan repositori GitHub Anda, arahkan ke URL Jenkins Anda.

Uji Coba

Lakukan perubahan pada kode, commit, dan push ke GitHub, lalu amati apakah build Jenkins terpicu secara otomatis.

Uji Coba Webhook

Lakukan Perubahan Kode

Buka repository lokal Anda dan lakukan perubahan kecil pada kode, misalnya mengubah pesan "Hello World" atau menambahkan komentar.

Commit dan Push

Commit perubahan Anda dengan perintah `git add .`, `git commit -m "Uji coba Webhook"`, dan `git push origin main` untuk mengirim perubahan ke GitHub.

Amati Jenkins

Buka dashboard Jenkins dan periksa apakah build baru terpicu secara otomatis. Ini menunjukkan bahwa Webhook berfungsi dengan baik.

Periksa Console Output

Klik pada build yang sedang berjalan dan periksa "Console Output" untuk melihat detail proses build dan memverifikasi bahwa semua tahap berjalan dengan benar.

Menambahkan Stage Build/Test

Untuk Spring Boot

Jika Anda menggunakan aplikasi Spring Boot, Anda dapat menambahkan stage build dan test yang lebih lengkap ke Jenkinsfile Anda. Ini akan menggunakan Maven untuk membangun aplikasi dan menjalankan pengujian unit.

```
stage('Build') { steps { sh 'mvn clean package -DskipTests' }}stage('Test') { steps { sh 'mvn test' } post { always { junit '**/target/surefire-reports/*.xml' } }}
```

Untuk Web Statis

Untuk aplikasi web statis, Anda dapat menambahkan validasi HTML atau linting JavaScript sebagai bagian dari proses build. Ini memerlukan alat tambahan yang diinstal di Jenkins.

```
stage('Validate') { steps { sh 'npm install' sh 'npm run lint' }}
```

Masalah Umum: Webhook Tidak Memicu Build

URL Webhook Salah

Pastikan URL Webhook di GitHub benar dan mengarah ke endpoint yang tepat di Jenkins (/github-webhook/).

Jenkins Tidak Dapat Diakses

Jika Jenkins berada di jaringan lokal, GitHub tidak dapat mengirim Webhook. Gunakan solusi seperti ngrok untuk membuat Jenkins dapat diakses dari internet.

Event Webhook Salah

Pastikan Anda memilih event yang benar di konfigurasi Webhook GitHub (biasanya "Just the push event").

Trigger Jenkins Tidak Aktif

Verifikasi bahwa "GitHub hook trigger for GITScm polling" dicentang di konfigurasi job Jenkins.

Tips untuk Integrasi **Jenkins-GitHub** yang Sukses

Amankan Webhook dengan Secret

Selalu gunakan webhook secret untuk memverifikasi bahwa permintaan benar-benar berasal dari GitHub. Ini mencegah serangan man-in-the-middle dan pemalsuan permintaan.

Mulai dengan Pipeline Sederhana

Mulai dengan pipeline Jenkins yang sangat sederhana dan pastikan integrasi webhook berfungsi sebelum menambahkan kompleksitas. Ini memudahkan isolasi masalah.

Periksa Log Jenkins

Jika Anda mengalami masalah, periksa log Jenkins (Manage Jenkins -> System Log) untuk informasi diagnostik yang dapat membantu mengidentifikasi akar masalah.

Cadangkan Konfigurasi Jenkins

Secara berkala cadangkan konfigurasi Jenkins Anda untuk mencegah kehilangan pengaturan penting jika terjadi kegagalan sistem.

A background image showing three business professionals in an office. A man in a suit and glasses is standing and clapping, while two women are seated at a desk, also clapping and smiling. The image is overlaid with a dark teal gradient.

</Rakamin

Terima kasih